



REPUBLIQUE DU SENEGAL



Un Peuple - Un But - une Foi

MINISTRE DE L'ENSEIGNEMENT SUPERIEUR
DE LA RECHERCHE, ET DE L'INNOVATION

RAPPORT:

FLASK

Présenté par Mouhamadou Falilou Sarr

Sous la direction de Mr OUYA

Septembre 2023

Introduction

Dans le paysage dynamique du développement web, la gestion efficace des données constitue un pilier fondamental pour la conception de systèmes robustes et évolutifs. Ce rapport explore de manière approfondie l'intégration de MySQL, un système de gestion de base de données relationnelle, avec les technologies de développement web telles que PHP, Flask, et les API (Interfaces de Programmation d'Application). Ces éléments convergent pour former une architecture puissante permettant la création, la lecture, la mise à jour et la suppression (CRUD) des données, éléments essentiels dans le développement d'applications web interactives et fonctionnelles.

I- INSTALLATION DES OUTILS



Vérifions si le paquet **MySQL-server** a été installé dans notre machine avec la commande **apt policy mysql-server** :

```
falilou@fs-ubuntu:~$ apt policy mysql-server
mysql-server:
  Installé : 8.0.35-0ubuntu0.20.04.1
  Candidat : 8.0.35-0ubuntu0.20.04.1
  Table de version :
  *** 8.0.35-0ubuntu0.20.04.1 500
      500 http://sn.archive.ubuntu.com/ubuntu focal-updates/main amd64 Packages
      500 http://sn.archive.ubuntu.com/ubuntu focal-updates/main i386 Packages
      500 http://security.ubuntu.com/ubuntu focal-security/main amd64 Packages
      500 http://security.ubuntu.com/ubuntu focal-security/main i386 Packages
      100 /var/lib/dpkg/status
  8.0.19-0ubuntu5 500
      500 http://sn.archive.ubuntu.com/ubuntu focal/main amd64 Packages
      500 http://sn.archive.ubuntu.com/ubuntu focal/main i386 Packages
falilou@fs-ubuntu:~$
```



Vérifions si le paquet **python3** a été installé dans notre machine avec la commande **apt policy python3** :

```
falilou@fs-ubuntu:~$ apt policy python3
python3:
  Installé : 3.8.2-0ubuntu2
  Candidat : 3.8.2-0ubuntu2
  Table de version :
  *** 3.8.2-0ubuntu2 500
      500 http://sn.archive.ubuntu.com/ubuntu focal/main amd64 Packages
      100 /var/lib/dpkg/status
falilou@fs-ubuntu:~$
```

Vérifions si le paquet **python3-pip** a été installé dans notre machine avec la commande **apt policy python3-pip** :

```

falilou@fs-ubuntu:~$ apt policy python3-pip
python3-pip:
  Installé : 20.0.2-5ubuntu1.10
  Candidat : 20.0.2-5ubuntu1.10
  Table de version :
  *** 20.0.2-5ubuntu1.10 500
        500 http://sn.archive.ubuntu.com/ubuntu focal-updates/universe amd64 Packages
        500 http://sn.archive.ubuntu.com/ubuntu focal-updates/universe i386 Packages
        500 http://security.ubuntu.com/ubuntu focal-security/universe amd64 Packages
        500 http://security.ubuntu.com/ubuntu focal-security/universe i386 Packages
        100 /var/lib/dpkg/status
  20.0.2-5ubuntu1 500
        500 http://sn.archive.ubuntu.com/ubuntu focal/universe amd64 Packages
        500 http://sn.archive.ubuntu.com/ubuntu focal/universe i386 Packages
falilou@fs-ubuntu:~$

```

Installons le paquet **pymysql** avec la commande **sudo apt install python3-pymysql** :

```

root@fs-ubuntu:/home/falilou/Bureau/apicrud# sudo apt install python3-pymysql
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances
Lecture des informations d'état... Fait
Paquets suggérés :
  python-pymysql-doc
Les NOUVEAUX paquets suivants seront installés :
  python3-pymysql
0 mis à jour, 1 nouvellement installés, 0 à enlever et 237 non mis à jour.
Il est nécessaire de prendre 38,9 ko dans les archives.
Après cette opération, 179 ko d'espace disque supplémentaires seront utilisés.
Réception de :1 http://sn.archive.ubuntu.com/ubuntu focal/main amd64 python3-pymysql all 0.9.3-2ubuntu3 [38,9 kB]
38,9 ko réceptionnés en 0s (123 ko/s)
Sélection du paquet python3-pymysql précédemment désélectionné.
(Lecture de la base de données... 242943 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../python3-pymysql_0.9.3-2ubuntu3_all.deb ...
Dépaquetage de python3-pymysql (0.9.3-2ubuntu3) ...
Paramétrage de python3-pymysql (0.9.3-2ubuntu3) ...
root@fs-ubuntu:/home/falilou/Bureau/apicrud#

```

Installons le paquet **flask** avec la commande **pip3 install flask** :

```

root@fs-ubuntu:/home/falilou# pip3 install flask
Collecting flask
  Downloading flask-3.0.0-py3-none-any.whl (99 kB)
    | 99 kB 720 kB/s
Collecting click>=8.1.3
  Downloading click-8.1.7-py3-none-any.whl (97 kB)
    | 97 kB 1.7 MB/s
Collecting itsdangerous>=2.1.2
  Downloading itsdangerous-2.1.2-py3-none-any.whl (15 kB)
Collecting Werkzeug>=3.0.0
  Downloading werkzeug-3.0.1-py3-none-any.whl (226 kB)
    | 226 kB 5.0 MB/s
Collecting importlib-metadata>=3.6.0; python_version < "3.10"
  Downloading importlib_metadata-6.8.0-py3-none-any.whl (22 kB)
Collecting blinker>=1.6.2
  Downloading blinker-1.7.0-py3-none-any.whl (13 kB)
Collecting Jinja2>=3.1.2
  Downloading Jinja2-3.1.2-py3-none-any.whl (133 kB)
    | 133 kB 8.0 MB/s
Collecting MarkupSafe>=2.1.1
  Downloading MarkupSafe-2.1.3-cp38-cp38-manylinux_2_17_x86_64_manylinux2014_x86_64.whl (25 kB)
Collecting zipp>=0.5
  Downloading zipp-3.17.0-py3-none-any.whl (7.4 kB)
Installing collected packages: click, itsdangerous, MarkupSafe, Werkzeug, zipp, importlib-metadata, blinker, Jinja2, flask
Attempting uninstall: click
  Found existing installation: Click 7.0
  Not uninstalling click at /usr/lib/python3/dist-packages, outside environment /usr
  Can't uninstall 'Click'. No files were found to uninstall.
Attempting uninstall: MarkupSafe
  Found existing installation: MarkupSafe 1.1.0
  Not uninstalling markupsafe at /usr/lib/python3/dist-packages, outside environment /usr
  Can't uninstall 'MarkupSafe'. No files were found to uninstall.
Attempting uninstall: blinker
  Found existing installation: blinker 1.4
  Not uninstalling blinker at /usr/lib/python3/dist-packages, outside environment /usr
  Can't uninstall 'blinker'. No files were found to uninstall.
Successfully installed Jinja2-3.1.2 MarkupSafe-2.1.3 Werkzeug-3.0.1 blinker-1.7.0 click-8.1.7 flask-3.0.0 importlib-metadata-6.8.0 itsdangerous-2.1.2 zipp-3.17.0
root@fs-ubuntu:/home/falilou#

```


Installons le paquet **flask-Restx** avec la commande **pip3 install flask-Restx** :

```
root@fs-ubuntu:/home/falilou# pip3 install flask-restx
Collecting flask-restx
  Downloading flask_restx-1.2.0-py2.py3-none-any.whl (2.8 MB)
    | 2.8 MB 648 kB/s
Collecting werkzeug<2.0.0,<3.0.0
  Downloading werkzeug-2.3.8-py3-none-any.whl (242 kB)
    | 242 kB 2.5 MB/s
Requirement already satisfied: pytz in /usr/lib/python3/dist-packages (from flask-restx) (2019.3)
Collecting jsonschema<4.17.3
  Downloading jsonschema-4.17.3-py3-none-any.whl (90 kB)
    | 90 kB 2.3 MB/s
Collecting aniso8601<=0.82
  Downloading aniso8601-9.0.1-py2.py3-none-any.whl (52 kB)
    | 52 kB 436 kB/s
Collecting importlib-resources
  Downloading importlib_resources-6.1.1-py3-none-any.whl (33 kB)
Collecting Flask<2.0.0,<3.0.0,>=0.8
  Downloading Flask-2.3.3-py3-none-any.whl (96 kB)
    | 96 kB 2.0 MB/s
Requirement already satisfied: MarkupSafe<=2.1.1 in /usr/local/lib/python3.8/dist-packages (from werkzeug<2.0.0,<3.0.0->flask-restx) (2.1.3)
Collecting attrs<=17.4.0
  Downloading attrs-23.1.0-py3-none-any.whl (61 kB)
    | 61 kB 2.2 MB/s
Collecting pkgutil_resolve_name<=1.3.10; python_version < "3.9"
  Downloading pkgutil_resolve_name-1.3.10-py3-none-any.whl (4.7 kB)
Collecting pyrsistent<=0.17.0,!=0.17.1,!=0.17.2,>=0.14.0
  Downloading pyrsistent-0.20.0-cp38-cp38-manylinux_2_17_x86_64_manylinux2014_x86_64.whl (121 kB)
    | 121 kB 2.7 MB/s
Requirement already satisfied: zipp<=3.1.0; python_version < "3.10" in /usr/local/lib/python3.8/dist-packages (from importlib-resources->flask-restx) (3.17.0)
Requirement already satisfied: click<8.1.3 in /usr/local/lib/python3.8/dist-packages (from Flask<2.0.0,<3.0.0,>=0.8->flask-restx) (8.1.7)
Requirement already satisfied: itsdangerous<=2.1.2 in /usr/local/lib/python3.8/dist-packages (from Flask<2.0.0,<3.0.0,>=0.8->flask-restx) (2.1.2)
Requirement already satisfied: Jinja2<=3.1.2 in /usr/local/lib/python3.8/dist-packages (from Flask<2.0.0,<3.0.0,>=0.8->flask-restx) (3.1.2)
Requirement already satisfied: blinker<=1.6.2 in /usr/local/lib/python3.8/dist-packages (from Flask<2.0.0,<3.0.0,>=0.8->flask-restx) (1.7.0)
Requirement already satisfied: importlib-metadata<=3.6.0; python_version < "3.10" in /usr/local/lib/python3.8/dist-packages (from Flask<2.0.0,<3.0.0,>=0.8->flask-restx) (6.8.0)
Installing collected packages: werkzeug, attrs, pkgutil_resolve_name, pyrsistent, importlib-resources, jsonschema, aniso8601, Flask, flask-restx
  Attempting uninstall: werkzeug
    Found existing installation: werkzeug 3.0.1
    Uninstalling werkzeug-3.0.1:
      Successfully uninstalled werkzeug-3.0.1
  Attempting uninstall: Flask
    Found existing installation: Flask 3.0.0
    Uninstalling Flask-3.0.0:
      Successfully uninstalled Flask-3.0.0
Successfully installed Flask-2.3.3 aniso8601-9.0.1 attrs-23.1.0 flask-restx-1.2.0 importlib-resources-6.1.1 jsonschema-4.17.3 pkgutil_resolve_name-1.3.10 pyrsistent-0.20.0 werkzeug-2.3.8
root@fs-ubuntu:/home/falilou#
```

Installons le paquet **jsonify** avec la commande **pip3 install jsonify** :

```
Successfully built jsonify
Installing collected packages: jsonify
Successfully installed jsonify-0.5
root@fs-ubuntu:/home/falilou#
```

Installons le paquet **crud** avec la commande **pip install crud** :

```
root@fs-ubuntu:/home/falilou/Bureau/apicrud# pip install crud
Collecting crud
  Downloading crud-0.1.tar.gz (1.1 kB)
Building wheels for collected packages: crud
  Building wheel for crud (setup.py) ... done
  Created wheel for crud: filename=crud-0.1-py3-none-any.whl size=1602 sha256=2edbcfdbf51a4feeb594c0c1f5dc30e021e3082d5eefa862d0fa3885a266cf58
  Stored in directory: /root/.cache/pip/wheels/02/81/30/41442a5ca423b58af86039f78fc967835b2f87f15ea4b69790
Successfully built crud
Installing collected packages: crud
Successfully installed crud-0.1
root@fs-ubuntu:/home/falilou/Bureau/apicrud#
```

II- CRÉATION DE LA BASE DE DONNÉE



On se connecte au serveur en utilisant le programme client par la commande :

mysql -u root -p

```
falilou@fs-ubuntu:~$ sudo -s
[sudo] Mot de passe de falilou :
root@fs-ubuntu:/home/falilou# mysql -u root -
ERROR 1049 (42000): Unknown database '-'
root@fs-ubuntu:/home/falilou# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 12
Server version: 8.0.34-0ubuntu0.20.04.1 (Ubuntu)

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> █
```



Création de la base de donnée **banque** :

```
mysql> create database banque;
Query OK, 1 row affected (0,01 sec)

mysql> use banque
Database changed
mysql> █
```



Créons la table **client** :

```
mysql> create table client(id int primary key auto_increment, prenom varchar(30), nom varchar(30), code varchar(4), numcompte varchar(40), solde varchar(30));
Query OK, 0 rows affected (0,07 sec)
```

```
mysql> █
```



Regardons la description de la table client :

```
mysql> desc client;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int  | NO   | PRI | NULL    | auto_increment |
| prenom | varchar(30) | YES |     | NULL    |               |
| nom   | varchar(30) | YES |     | NULL    |               |
| code  | varchar(4)  | YES |     | NULL    |               |
| numcompte | varchar(40) | YES |     | NULL    |               |
| solde | varchar(30) | YES |     | NULL    |               |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0,00 sec)

mysql> █
```

Créons l'utilisateur falilou :

```
mysql> create user 'falilou'@'localhost' identified by 'passer';
Query OK, 0 rows affected (0,02 sec)

mysql> █
```

Donnons tous les privilèges à l'utilisateur dans la base de donnée banque :

```
mysql> grant all privileges on banque.* to falilou@localhost;
Query OK, 0 rows affected (0,01 sec)

mysql> █
```

Appliquons les privilèges :

```
mysql> flush privileges;
Query OK, 0 rows affected (0,01 sec)
```

III- CRÉATION DE L'API AVEC FLASK

ACTIVITÉ 1 : Création d'un programme python pour créer des comptes bancaires qui sera relia à une page web

Ecrivons le programme python pour créer des comptes bancaires qui seront ajoutés dans notre base de donnée banque :

```
GNU nano 4.8
from flask import Flask, request, jsonify
from flask_restx import Api, Resource
import crud

banque = Flask(__name__)
client = Api(banque)

@client.route("/voir")
class Voir(Resource):
    def get(self):
        resultat = crud.lecture()
        return jsonify(resultat)

@client.route('/creation/<prenom>/<nom>/<code>/<numcompte>/<solde>')
class Creation(Resource):
    def post(self, prenom, nom, code, numcompte, solde):
        crud.creation(prenom, nom, code, numcompte, solde)
        return {"message": f"Le compte {numcompte} a été ajouté(e) avec succès "}

if __name__ == '__main__':
    banque.run(host="127.0.0.1", port=5000)
```

```
GNU nano 4.8 crud.py
from flask import jsonify

comptes = []

def creation(prenom, nom, code, numcompte, solde):
    compte = {
        "prenom": prenom,
        "nom": nom,
        "code": code,
        "numcompte": numcompte,
        "solde": float(solde)
    }
    comptes.append(compte)
```

POST /creation/{prenom}/{nom}/{code}/{numcompte}/{solde}

Parameters
Cancel

Name	Description
prenom * required string (path)	Bamba
nom * required string (path)	Wade
code * required string (path)	2407
numcompte * required string (path)	1000
solde * required string (path)	100000

Execute
Clear

Responses
Response content type application/json

Curl

```
curl -X 'POST' \
  'http://192.168.1.180:5000/creation/Bamba/Wade/2407/1000/100000' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://192.168.1.180:5000/creation/Bamba/Wade/2407/1000/100000
```

Server response

Code	Details
200	Response body <pre>{ "message": "Le compte 1000 a été ajouté avec succès " }</pre> Download Response headers <pre>connection: close content-length: 75 content-type: application/json date: Thu, 30 Nov 2023 12:26:57 GMT server: Werkzeug/2.3.8 Python/3.8.10</pre>

Responses

Code	Description
200	Success

POST

/creation/{prenom}/{nom}/{code}/{numcompte}/{solde}

Cancel

Parameters

Name	Description
prenom * required string (path)	Samuel
nom * required string (path)	Ouya
code * required string (path)	1234
numcompte * required string (path)	1002
solde * required string (path)	45000

ExecuteClear

Responses

Response content typeapplication/json

Curl

curl -X 'POST' \
'http://192.168.1.180:5000/creation/Samuel/Ouya/1234/1002/45000' \
-H 'accept: application/json' \
-d ''

Request URL

http://192.168.1.180:5000/creation/Samuel/Ouya/1234/1002/45000

Server response

Code	Details
200	Response body { "message": "Le compte 1002 a été ajouté avec succès " } Download

POST

/creation/{prenom}/{nom}/{code}/{numcompte}/{solde}

Cancel

Parameters

Name	Description
prenom * required string (path)	Falliou
nom * required string (path)	Sarr
code * required string (path)	1301
numcompte * required string (path)	1001
solde * required string (path)	500000

ExecuteClear

Responses

Response content typeapplication/json

Curl

curl -X 'POST' \
'http://192.168.1.180:5000/creation/Falliou/Sarr/1301/1001/500000' \
-H 'accept: application/json' \
-d ''

Request URL

http://192.168.1.180:5000/creation/Falliou/Sarr/1301/1001/500000

Server response

Code	Details
200	Response body { "message": "Le compte 1001 a été ajouté avec succès " } Download

Vérifions dans notre base de donnée si les comptes ont bien été ajoutés :

```
mysql> select * from client;
+----+-----+-----+-----+-----+-----+
| id | prenom | nom   | code | numcompte | solde |
+----+-----+-----+-----+-----+-----+
| 1  | Bamba  | Wade | 2407 | 1000      | 100000 |
| 3  | Samuel | Ouya   | 1234 | 1002      | 45000  |
| 5  | Falilou | Sarr  | 1301 | 1001      | 500000 |
+----+-----+-----+-----+-----+-----+
3 rows in set (0,00 sec)

mysql> 
```

ACTIVITÉ 2 : Création d'un programme python modifier les soldes des comptes bancaires

Ajoutons le code python pour modifier les soldes des comptes :

```
client.route('/update/<prenom>/<nom>/<numcompte>/<solde>')
class Update(Resource):
    def put(self, prenom, nom, numcompte, solde):
        success = crud.update(prenom, nom, numcompte, solde)
        if success:
            return jsonify({"message": f"Le compte {numcompte} a été mise à jour"})
        else:
            return jsonify({"message": f"Erreur lors de la mise a jour du compte"})
```

Ecrivons le code python crud :

```
def update(prenom, nom, numcompte, solde):
    database()
    sql = "update client set solde=%s where prenom=%s and nom=%s and numcompte=%s"
    val = (prenom, nom, numcompte, solde)
    curseur.execute(sql, val)
    conn.commit()
    return "mise a jour reussie"
```

Modifions le solde du compte de Falilou Sarr :

PUT
/update/{prenom}/{nom}/{numcompte}/{solde}

Parameters
Cancel

Name	Description
prenom * required string (path)	Falilou
nom * required string (path)	Sarr
numcompte * required string (path)	1001
solde * required string (path)	450000

Execute
Clear

Responses
Response content type application/json

Curl

```
curl -X 'PUT' \
'http://192.168.1.180:5000/update/Falilou/Sarr/1001/450000' \
-H 'accept: application/json'
```

Request URL

```
http://192.168.1.180:5000/update/Falilou/Sarr/1001/450000
```

Server response

Code	Details
200	Response body <pre>{ "message": "Le compte 1001 a été mise à jour" }</pre> Download

Vérifions si le solde a été mise à jour :

```
mysql> select * from client;
+----+-----+-----+-----+-----+-----+
| id | prenom | nom | code | numcompte | solde |
+----+-----+-----+-----+-----+-----+
| 3 | Samuel | Ouya | 1234 | 1002 | 150000 |
| 5 | Falilou | Sarr | 1301 | 1001 | 450000 |
+----+-----+-----+-----+-----+-----+
2 rows in set (0,00 sec)

mysql> 
```

ACTIVITÉ 3 : Création d'un programme python faieres des versements dans les comptes bancaires



Ajoutons le code python pour effectuer des versements en entrant le code et le montant à verser dans notre fichier python :

```
@client.route('/versement/<numcompte>/<code>/<montant>')
class Versement(Resource):
    def post(self, numcompte, code, montant):
        success = crud.versement(numcompte, code, montant)
        if success:
            return jsonify({"message": f"Versement de {montant} effectué avec succès sur le compte {numcompte}"})
        else:
            return jsonify({"message": f"Erreur lors du versement sur le compte {numcompte}"})
```



Ecrivons le code python crud :

```
def versement(numcompte, code, montant):
    database()
    sql = "update client set solde=solde + %s where numcompte=%s and code=%s"
    val = (float(montant), numcompte ,code)
    curseur.execute(sql, val)
    conn.commit()
    return "Versement de {montant} effectué avec succès sur le compte {code}"
```



Effectuons un versement de 50000 sur le compte de Falilou Sarr :

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** /versement/{numcompte}/{code}/{montant}
- Parameters:**
 - numcompte** (required, string, path): 1001
 - code** (required, string, path): 1301
 - montant** (required, string, path): 50000
- Buttons:** Execute, Clear
- Responses:** Response content type: application/json
- Curl:**

```
curl -X 'POST' \
  'http://192.168.1.180:5000/versement/1001/1301/50000' \
  -H 'accept: application/json' \
  -d ''
```
- Request URL:** http://192.168.1.180:5000/versement/1001/1301/50000
- Server response:**
 - Code:** 200
 - Details:** Response body: {"message": "Versement de 50000 effectué avec succès sur le compte 1001"}
- Buttons:** Download

Vérifions si le versement a bien été fait sur le compte dans notre base de donnée :

```
mysql> select * from client;
```

id	prenom	nom	code	numcompte	solde
1	Bamba	Wade	2407	1000	100000
3	Samuel	Ouya	1234	1002	45000
5	Falilou	Sarr	1301	1001	500000

3 rows in set (0,00 sec)

avant

```
mysql> select * from client;
```

id	prenom	nom	code	numcompte	solde
1	Bamba	Wade	2407	1000	100000
3	Samuel	Ouya	1234	1002	45000
5	Falilou	Sarr	1301	1001	550000

3 rows in set (0,00 sec)

après

ACTIVITÉ 3 : Création d'un programme python pour faire des retraits dans les comptes bancaires

Ajoutons le code python pour faire des retraits d'argent :

```
@client.route('/retrait/<numcompte>/<code>/<montant>')
class Retrait(Resource):
    def post(self, numcompte, code, montant):
        success = crud.retrait(numcompte, code, montant)
        if success:
            return jsonify({"message": f"Retrait de {montant} effectué avec succès sur le compte {numcompte}"})
        else:
            return jsonify({"message": f"Retrait impossible"})
```

Ajoutons le code python crud :

```
def retrait(numcompte, code, montant):
    database()
    sql = "update client set solde=solde - %s where numcompte=%s and code=%s"
    val = (float(montant), numcompte, code)
    curseur.execute(sql, val)
    conn.commit()
    return "Retrait de {montant} effectué avec succès sur le compte {numcompte}"
```


Effectuons un retrait de 25000 sur le compte de Bamba Wade :

POST /retrait/{numcompte}/{code}/{montant}

Parameters

Name

Description

numcompte * required

string (path)

1000

code * required

string (path)

2407

montant * required

string (path)

25000

Execute

Clear

Responses

Response content type application/json

Curl

```
curl -X 'POST' \
  'http://192.168.1.180:5000/retrait/1000/2407/25000' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

http://192.168.1.180:5000/retrait/1000/2407/25000

Server response

Code

Details

200

Response body

```
{
  "message": "Retrait de 25000 effectué avec succès sur le compte 1000"
}
```

Download



Vérifions si le retrait est bien fait dans la base de donnée :

```
mysql> select * from client;
```

id	prenom	nom	code	numcompte	solde
1	Bamba	Wade	2407	1000	100000
3	Samuel	Ouya	1234	1002	45000
5	Falilou	Sarr	1301	1001	550000

3 rows in set (0,00 sec)

avant

```
mysql> select * from client;
```

id	prenom	nom	code	numcompte	solde
1	Bamba	Wade	2407	1000	75000
3	Samuel	Ouya	1234	1002	45000
5	Falilou	Sarr	1301	1001	550000

3 rows in set (0,00 sec)

après

ACTIVITÉ 4 : Création d'un programme python pour faire des transferts entre les comptes bancaires



Ajoutons le code python pour faire des transferts d'argent via les comptes :

```
@client.route('/transfert/<numcompte_donneur>/<numcompte_reveur>/<montant>')
class Transfert(Resource):
    def post(self, numcompte_donneur, numcompte_reveur, montant):
        success = crud.transfert(numcompte_donneur, numcompte_reveur, montant)
        if success:
            return jsonify({"message": f"Transfert reussie"})
        else:
            return jsonify({"message": f"Transfert impossible"})
```



Ajoutons le code python crud :

```
def transfert(numcompte_donneur, numcompte_reveur, montant):
    database()
    sql_retrait = "update client set solde = solde - %s where numcompte = %s"
    val_retrait = (float(montant), numcompte_donneur)
    curseur.execute(sql_retrait, val_retrait)
    sql_depote = "update client set solde = solde + %s where numcompte = %s"
    val_depote = (float(montant), numcompte_reveur)
    curseur.execute(sql_depote, val_depote)
    conn.commit()
    return "Transfert de {montant} effectué avec succès"
```

NB : Contrairement aux autres programmes crud ici on doit faire deux requêtes, la première pour retirer de l'argent du solde du donneur et la deuxième pour envoyer au receveur l'argent retirer au donneur.



Effectuons un transfert de 100000 du compte de Falilou Sarr vers Samuel Ouya :

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** /transfert/{numcompte_donneur}/{numcompte_reveur}/{montant}
- Parameters:**
 - numcompte_donneur** (string, required, path): 1001
 - numcompte_reveur** (string, required, path): 1002
 - montant** (string, required, path): 100000
- Buttons:** Execute, Clear
- Responses:**
 - Response content type:** application/json
 - Curl:**

```
curl -X 'POST' \
  'http://192.168.1.180:5000/transfert/1001/1002/100000' \
  -H 'accept: application/json' \
  -d ''
```
 - Request URL:** http://192.168.1.180:5000/transfert/1001/1002/100000
 - Server response:**
 - Code:** 200
 - Details:** Response body
 - Response body:**

```
{
  "message": "Transfert reussie"
}
```
 - Buttons:** Download

Vérifions les soldes des comptes dans la base de donnée :

```
mysql> select * from client;
```

id	prenom	nom	code	numcompte	solde
1	Bamba	Wade	2407	1000	75000
3	Samuel	Ouya	1234	1002	45000
5	Falilou	Sarr	1301	1001	550000

3 rows in set (0,00 sec)

avant

```
mysql> select * from client;
```

id	prenom	nom	code	numcompte	solde
1	Bamba	Wade	2407	1000	75000
3	Samuel	Ouya	1234	1002	145000
5	Falilou	Sarr	1301	1001	450000

3 rows in set (0,00 sec)

après

```
mysql>
```

ACTIVITÉ 4 : Création d'un programme python pour consulter les soldes des comptes bancaires

Ajoutons le code python pour consulter les soldes des comptes :

```
@client.route('/consultation/<numcompte>/<code>')
class Consultation(Resource):
    def get(self, numcompte, code):
        solde = crud.consultation(numcompte, code, solde=None)
        if solde is not None:
            return jsonify({"message": f"Le solde du compte {numcompte} est : {solde}"})
        else:
            return jsonify({"message": f"Consultation impossible"})
```

Ajoutons le code python crud :

```
def consultation(numcompte, code, solde):
    database()
    sql = "select solde from client where numcompte=%s and code=%s "
    val = (numcompte, code)
    curseur.execute(sql, val)
    result= curseur.fetchone()
    return result[0]
```

Consultons le solde du compte bancaire de Bamba Wade avec 1000 comme numéro de compte :

GET /consultation/{numcompte}/{code}

Parameters

Name	Description
numcompte * required string (path)	1000
code * required string (path)	2407

Execute Clear

Responses

Response content type application/json

Curl

```
curl -X 'GET' \
'http://192.168.1.180:5000/consultation/1000/2407' \
-H 'accept: application/json'
```

Request URL

```
http://192.168.1.180:5000/consultation/1000/2407
```

Server response

Code	Details
200	Response body <pre>{ "message": "Le solde du compte 1000 est : 75000" }</pre> Download

Response headers

ACTIVITÉ 5 : Création d'un programme python pour supprimer des comptes bancaires

Ajoutons le code python pour supprimer des comptes :

```
@client.route('/suppression/<prenom>/<nom>/<numcompte>/<code>')
class Suppression(Resource):
    def delete(self, prenom, nom, numcompte, code):
        success, message = crud.suppression(prenom, nom, numcompte, code)
        if success:
            return jsonify({"message": f"Le compte de {numcompte} est supprime"})
        else:
            return jsonify({"message": f"Suppression impossible"})
```

Ajoutons le code python crud :

```
def suppression(prenom, nom, numcompte, code):
    database()
    sql = "delete from client where prenom=%s and nom=%s and numcompte=%s and code=%s "
    val = (prenom, nom, numcompte, code)
    curseur.execute(sql, val)
    conn.commit()
    return "Le compte de {prenom} {nom} a été supprime"
```

Supprimons un compte bancaire de Bamba Wade avec 1000 comme numéro de compte :

DELETE /suppression/{prenom}/{nom}/{numcompte}/{code}

Parameters

Name

Description

prenom * required

string

(path)

Bamba

nom * required

string

(path)

Wade

numcompte * required

string

(path)

1000

code * required

string

(path)

2407

Execute

Clear

Responses

Response content type application/json

Curl

```
curl -X 'DELETE' \
'http://192.168.1.180:5000/suppression/Bamba/Wade/1000/2407' \
-H 'accept: application/json'
```

Request URL

http://192.168.1.180:5000/suppression/Bamba/Wade/1000/2407

Server response

Code

Details

200

Response body

```
{
  "message": "Le compte de 1000 est supprimé avec succès"
}
```

Download

Vérifions que le compte a bien été supprimé dans la base de donnée :

```
mysql> select * from client;
+----+-----+-----+-----+-----+-----+-----+
| id | prenom | nom  | code | numcompte | solde |
+----+-----+-----+-----+-----+-----+
| 3  | Samuel | Ouya | 1234 | 1002      | 145000 |
| 5  | Falilou | Sarr | 1301 | 1001      | 450000 |
+----+-----+-----+-----+-----+-----+
2 rows in set (0,00 sec)

mysql>
```


III- HÉBERGEMENT D'API(WSGI) AVEC APACHE2



Installons le paquet `libapache2-mod-wsgi-py3` :

```
falilou@fs-ubuntu:/var/www/html/tp_api$ sudo -s
[sudo] Mot de passe de falilou :
root@fs-ubuntu:/var/www/html/tp_api# apt install libapache2-mod-wsgi-py3
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances
Lecture des informations d'état... Fait
Les NOUVEAUX paquets suivants seront installés :
  libapache2-mod-wsgi-py3
0 mis à jour, 1 nouvellement installés, 0 à enlever et 77 non mis à jour.
Il est nécessaire de prendre 93,8 ko dans les archives.
Après cette opération, 300 ko d'espace disque supplémentaires seront utilisés.
Réception de :1 http://sn.archive.ubuntu.com/ubuntu focal-updates/main amd64 libapache2-mod-wsgi-py3 amd64 4.6.8-1ubuntu3.1 [93,8 kB]
93,8 ko réceptionnés en 10s (9.797 o/s)
Sélection du paquet libapache2-mod-wsgi-py3 précédemment désélectionné.
(Lecture de la base de données... 271244 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../libapache2-mod-wsgi-py3_4.6.8-1ubuntu3.1_amd64.deb ...
Dépaquetage de libapache2-mod-wsgi-py3 (4.6.8-1ubuntu3.1) ...
Paramétrage de libapache2-mod-wsgi-py3 (4.6.8-1ubuntu3.1) ...
apache2_invoke: Enable module wsgi
root@fs-ubuntu:/var/www/html/tp_api#
```



Copions le projet `apicrud` dans `/var/www/html` :

```
root@fs-ubuntu:/home/falilou/Bureau# cp -r apicrud/ /var/www/html/
root@fs-ubuntu:/home/falilou/Bureau#
```

```
root@fs-ubuntu:/var/www/html# ls
apicrud  formulaire.html  index.html  modification.html  suppression.html  tp_api  webrtc-node-app
root@fs-ubuntu:/var/www/html#
```



Créons le fichier `__init__.py` :

```
root@fs-ubuntu:/var/www/html# touch __init__.py
root@fs-ubuntu:/var/www/html#
```



Copions l'api dans `__init__.py` :

```
root@fs-ubuntu:/var/www/html/apicrud# cp banque.py __init__.py
root@fs-ubuntu:/var/www/html/apicrud#
```



Donnons les droits d'exécution :

```
root@fs-ubuntu:/var/www/html/apicrud# chmod +x __init__.py
root@fs-ubuntu:/var/www/html/apicrud#
```



Créons et configurons le fichier virtuel de notre site avec la commande **nano** **/etc/apache2/sites-available/api.conf** :

```
GNU nano 4.8 api.conf
<VirtualHost *: 80>
    ServerName api.falilou.sn
    WSGIScriptAlias / /var/www/html/apicrud/banque.wsgi
</VirtualHost>
```



Vérification de syntaxe sur les fichiers de configuration avec la commande **apachectl -t** :

```
root@fs-ubuntu:/etc/apache2/sites-available# apachectl -t
AH00558: apache2: Could not reliably determine the server's fully qualified domain name,
Syntax OK
root@fs-ubuntu:/etc/apache2/sites-available#
```



Activons le site virtuel et rechargeons le service apache2 pour appliquer les modifications :

```
root@fs-ubuntu:/etc/apache2/sites-available# a2ensite api.conf
Enabling site api.
To activate the new configuration, you need to run:
  systemctl reload apache2
root@fs-ubuntu:/etc/apache2/sites-available# systemctl reload apache2
root@fs-ubuntu:/etc/apache2/sites-available#
```



Créons le script wsgi en créant et configurant le fichier banque.wsgi dans **/var/www/html/apicrud** :

```
GNU nano 4.8 banque.wsgi
import sys
sys.path.append('/var/www/html/apicrud')
sys.stdout=sys.stderr
from banque import banque as application
```



Donnons les droits d'exécution :

```
root@fs-ubuntu:/var/www/html/apicrud# nano banque.wsgi
root@fs-ubuntu:/var/www/html/apicrud# chmod +x banque.wsgi
root@fs-ubuntu:/var/www/html/apicrud#
```

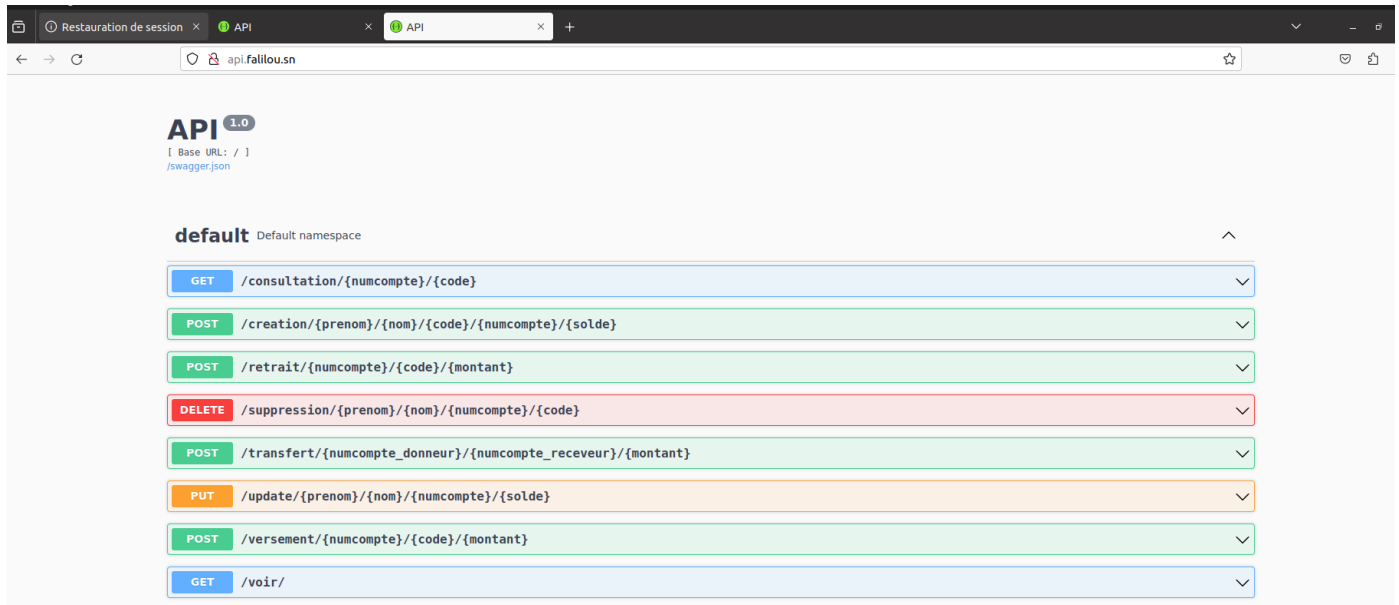


Ajoutons notre nom de domaine dans **/etc/hosts** :

```
GNU nano 4.8 /etc/hosts
127.0.0.1    localhost api.falilou.sn
127.0.1.1    UBUNTU.falilou.virtualbox.org  UBUNTU

# The following lines are desirable for IPv6 capable hosts
::1         ip6-localhost ip6-loopback
fe00::0     ip6-localnet
ff00::0     ip6-mcastprefix
ff02::1     ip6-allnodes
ff02::2     ip6-allrouters
```

Testons le site en tapant <http://api.falilou.sn/> dans le navigateur :



The screenshot shows a web browser window with the address bar displaying `api.falilou.sn`. The page content is the Swagger API documentation for 'API 1.0'. It features a 'default' namespace section with a list of endpoints, each with a colored button indicating the HTTP method and a dropdown arrow for details.

Method	Endpoint
GET	<code>/consultation/{numcompte}/{code}</code>
POST	<code>/creation/{prenom}/{nom}/{code}/{numcompte}/{solde}</code>
POST	<code>/retrait/{numcompte}/{code}/{montant}</code>
DELETE	<code>/suppression/{prenom}/{nom}/{numcompte}/{code}</code>
POST	<code>/transfert/{numcompte_donneur}/{numcompte_reveur}/{montant}</code>
PUT	<code>/update/{prenom}/{nom}/{numcompte}/{solde}</code>
POST	<code>/versement/{numcompte}/{code}/{montant}</code>
GET	<code>/voir/</code>